

Day 22: Limited Login Attempts

1. Day Number

Day 22

2. Topic Name

Loop with Counter

3. Connection

You already learned how to validate a password using if-else.

Today, you will improve that program by allowing the user to try the password only **three times**.

```
Password validation
+
While loop
+
Attempt counter
=
Limited login attempts
```

4. Important Topics

- while loop
- Counter variable
- Loop condition
- break
- if-else
- Comparison operator ==

5. Foundational Notes

What is a counter?

A counter is a variable that keeps track of how many times something has happened.

Example:

```
attempts = 0
```

After one attempt:

```
attempts = attempts + 1
```

The value changes like this:

```
0 → 1 → 2 → 3
```

Why use a while loop?

A while loop repeats code while its condition is True.

```
while attempts < 3:  
    # Ask for password
```

This means:

| Keep asking while fewer than three attempts have been used.

What does break do?

break immediately stops the loop.

For example, when the user enters the correct password:

```
if entered_password == saved_password:  
    print("Login successful")  
    break
```

Without break, the program may continue asking for the password even after a successful login.

6. Easy Example

Here is a simple counter example unrelated to the main problem:

```
count = 0  
  
while count < 3:  
    print("Welcome")  
    count = count + 1
```

Output:

```
Welcome  
Welcome  
Welcome
```

The loop stops when count becomes 3.

7. Problem Statement

Create a Python program that:

1. Stores a password.
2. Allows the user to enter the password up to three times.
3. Prints "Login successful" when the password is correct.
4. Stops asking after a correct password.
5. Prints "Account locked" when all three attempts are incorrect.

8. Concepts Used

Concept	Purpose
Variable	Store the correct password
<code>input()</code>	Read the entered password
while loop	Repeat the login process
Counter	Count failed attempts
if	Check whether the password matches
<code>==</code>	Compare two password strings
<code>break</code>	Exit after successful login
else	Handle an incorrect password

9. Thought Process

Step 1: Store the correct password

Create a variable containing the password that the user must enter.

Step 2: Create an attempt counter

Start the counter at 0.

```
attempts = 0
```

Step 3: Repeat while attempts are below three

Use a while loop that continues while the attempt counter is less than 3.

Step 4: Ask for the password

Inside the loop, take password input from the user.

Step 5: Check the password

Compare the entered password with the stored password.

- If they match:
- Print success.
- Stop the loop using `break`.
- If they do not match:
- Increase the attempt counter.
- Optionally print an incorrect-password message.

Step 6: Check whether all attempts failed

After three incorrect attempts, print:

```
Account locked
```

Be careful not to print this message when the user successfully logs in.

10. Pseudocode

```
START

Store the correct password

Set attempts to 0
Set maximum attempts to 3

WHILE attempts is less than maximum attempts

    Ask the user to enter password

    IF entered password equals correct password
        Print "Login successful"
        Stop the loop
    ELSE
        Increase attempts by 1
        Print "Wrong password"

IF attempts equals maximum attempts
    Print "Account locked"

END
```

11. Suggested Solving Approach: Functional Approach

Organize the login logic inside a function.

Possible structure:

```
Create a login function
    Store or receive the correct password
    Create the attempt counter
    Run the while loop
    Check each entered password
    Return or stop when login succeeds
    Show account locked after three failures

Call the login function
```

A possible function idea is:

```
def login(correct_password):
    # Login logic goes here
```

The function could return a Boolean value:

```
True  → login successful
False → account locked
```

12. Easy Edge Cases

Correct on the first try

```
Attempt 1: Correct password  
Result: Login successful
```

The program should stop immediately.

Correct on the third try

```
Attempt 1: Wrong  
Attempt 2: Wrong  
Attempt 3: Correct  
Result: Login successful
```

The program must not print "Account locked".

All attempts are wrong

```
Attempt 1: Wrong  
Attempt 2: Wrong  
Attempt 3: Wrong  
Result: Account locked
```

Password case difference

```
Saved password: Python123  
Entered password: python123
```

These are different because Python string comparison is case-sensitive.

13. Expected Input

Example 1:

```
Enter password: Python123
```

Example 2:

```
Enter password: wrong  
Enter password: incorrect  
Enter password: Python123
```

Example 3:

```
Enter password: wrong1  
Enter password: wrong2  
Enter password: wrong3
```

14. Expected Output

Correct first try

```
Enter password: Python123
Login successful
```

Correct third try

```
Enter password: wrong
Wrong password

Enter password: incorrect
Wrong password

Enter password: Python123
Login successful
```

All three attempts fail

```
Enter password: wrong1
Wrong password

Enter password: wrong2
Wrong password

Enter password: wrong3
Wrong password

Account locked
```

15. Hint Only

Create an attempts variable starting at 0. Use a while attempts < 3 loop. Increase the counter only when the password is incorrect, and use break when the password is correct.